Informe Técnico y Matriz de Pruebas: Seguridad JWT, Roles y Gestión de Usuarios

Universidad Peruana Unión (UPeU)
Facultad de Ingeniería y Arquitectura · Escuela Profesional de Ingeniería de Sistemas
Asignatura: Lenguaje de Programación II · Ciclo IV · Semestre 2026-2
Unidad II: Frontend SPA empresarial seguro
Sesión 10: Pruebas de seguridad y gestión de contraseñas en PharmaBackend (Reto 02)
Estudiante: Samuel Vilca
Docente: Reyna Barreto Benjamin David
Fecha: Octubre 2026

---

1. Información General y Metadatos de la Entrega

- **Título de la Actividad:** Matriz de pruebas de seguridad, cambio de contraseña, asignación de roles y desactivación de usuarios.
- **Objetivo de la Sesión:** Verificar sistemáticamente la autenticación JWT y la autorización por permisos de PharmaBackend contrastando resultados esperados y obtenidos, e implementar el cambio de contraseña con BCrypt, la asignación de roles (relación muchos a muchos) y la desactivación de usuarios respetando las reglas de seguridad.
- **Repositorio Backend:** PharmaBackend (Spring Boot, Java 21, Spring Security 6, JJWT 0.12.6).
- **Rama de Trabajo:** `feature/seguridad-pruebas-vilca` (creada a partir de `develop`).
- **Colección de Postman:** `postman/PharmaSoft-S10-Seguridad.json` con variable de entorno `baseUrl = http://localhost:8080/api/v1` y variables de tokens dinámicas.

---

2. Preparación del Entorno y Datos de Prueba

Para garantizar la reproducibilidad y validez de cada prueba, se aprovisionó la base de datos relacional con las siguientes precondiciones:

1. Cuentas de Usuario y Roles Registrados:

- **Administrador (`admin`):** Con rol `ADMIN` (ID 1). Posee los 7 permisos del sistema (`CATALOGO_LEER`, `CATALOGO_ESCRIBIR`, `CLIENTES_LEER`, `CLIENTES_ESCRIBIR`, `VENTAS_LEER`, `VENTAS_REGISTRAR`, `USUARIOS_ADMINISTRAR`, `REPORTES_VER`).
- **Vendedor (`vendedor`):** Con rol `VENDEDOR` (ID 2). Posee permisos de lectura de catálogo y clientes, y emisión de ventas (`VENTAS_REGISTRAR`).
- **Almacenero (`almacen`):** Con rol `ALMACENERO` (ID 3). Posee permisos de lectura y actualización de catálogo (`CATALOGO_LEER`, `CATALOGO_ESCRIBIR`).
- **Supervisora (`supervisor`):** Con roles múltiples `[VENDEDOR, ALMACENERO]`. Reúne la unión de permisos de catálogo y ventas.
- **Usuario Inactivo (`inactivo`):** Cuenta con `estado = false`, configurada para comprobar el rechazo inmediato en login.

2. Parámetros Criptográficos:

- **Algoritmo de Hashing:** BCrypt con factor de costo 10 y límite de procesamiento de 72 bytes.
- **Firma de Tokens JWT:** Clave simétrica HMAC-SHA (HS384) de 384 bits con expiración de 60 minutos (3600 segundos).

Parte A: Matriz de Pruebas de Seguridad (8 Puntos)

La siguiente matriz consolida los 28 casos de prueba ejecutados (16 casos base fijados en la guía académica, 4 casos propios de grupos distintos y 8 casos correspondientes a las nuevas operaciones de administración de contraseñas, roles y estado de usuarios B-01 a B-08).

ID	Grupo	Usuario / Token	Petición	Resultado Esperado	Resultado Obtenido	Estado	Evidencia
A-01	Autenticación	Sin token	POST /api/v1/auth/login (admin, vendedor, almacen, supervisor)	200 OK; cada respuesta trae sus roles y permisos; supervisora trae la unión de VENDEDOR y ALMACENERO	200 OK; tokens emitidos con roles y permisos correctos	PASA	A-01_postman.png
A-02	Autenticación	Sin token	POST /api/v1/auth/login	401 Unauthorized con el mismo	401 Unauthorized;	PASA	A-02_postman.png

ID	Grupo	Usuario / Token	Petición	Resultado Esperado	Resultado Obtenido	Estado	Evidencia
			con usuario inexistente y con contraseña incorrecta	mensaje en ambos casos: "Usuario o contraseña incorrectos"	mensaje idéntico sin revelar si la cuenta existe		
A-03	Autenticación	Sin token	POST /api/v1/auth/login con usuario inactivo y contraseña correcta	401 Unauthorized; mensaje uniforme para no revelar estado	401 Unauthorized; cuenta inactiva rechazada con mensaje idéntico	PASA	A-03_postman.png
A-04	Autenticación	Sin token	POST /api/v1/auth/login con usuario en mayúsculas (VENDEDOR)	200 OK con login case-insensitive según regla de usabilidad	200 OK; username resuelto en minúsculas en el token	PASA	A-04_postman.png
A-05	Autenticación (Propio)	Sin token	POST /api/v1/auth/login con campos vacíos ("" y "")	400 Bad Request; validación @NotBlank en username y password	400 Bad Request; errores estructurados devueltos por el backend	PASA	A-05_postman.png
T-01	Token	Token sin Bearer	GET /api/v1/productos con Authorization: `<token>` sin prefijo	401 Unauthorized: "Debe iniciar sesión para acceder a este recurso"	401 Unauthorized; filtro omite cabecera inválida y rechaza	PASA	T-01_postman.png
T-02	Token	Token alterado	GET /api/v1/productos con un carácter cambiado en el payload	401 Unauthorized: "El token no es válido"	401 Unauthorized; falla verificación de firma HMAC	PASA	T-02_postman.png
T-03	Token	Token otra clave	GET /api/v1/productos con token emitido bajo otro secreto	401 Unauthorized: "El token no es válido"	401 Unauthorized; signature mismatch	PASA	T-03_postman.png

ID	Grupo	Usuario / Token	Petición	Resultado Esperado	Resultado Obtenido	Estado	Evidencia
					capturado por JJWT		
T-04	Token	Token vencido	GET /api/v1/productos con token expirado (expiracion-minutos: 1)	401 Unauthorized: "El token ha expirado. Inicie sesión nuevamente"	401 Unauthorized; ExpiredJwtException mapeada correctamente	PASA	T-04_postman.png
T-05	Token	Token Vendedor	Decodificar en jwt.io y consultar GET /api/v1/auth/me	Claims sub, authorities, iat, exp correctos; ningún dato sensible	Claims válidos; sin contraseñas ni hashes expuestos	PASA	T-05_postman.png
T-06	Token (Propio)	Sin cabecera	GET /api/v1/ventas sin cabecera Authorization	401 Unauthorized: "Debe iniciar sesión para acceder a este recurso"	401 Unauthorized; RestAuthenticationEntryPoint responde	PASA	T-06_postman.png
P-01	Permisos	VENDEDOR	POST, PUT y DELETE sobre /api/v1/categorias	403 Forbidden en los tres intentos (requiere CATALOGO_ESCRIBIR)	403 Forbidden: "No tiene permisos para realizar esta operación"	PASA	P-01_postman.png
P-02	Permisos	ALMACENERO	GET /api/v1/clientes y POST /api/v1/ventas	403 Forbidden en ambos (carece de CLIENTES_LEER y VENTAS_REGISTRAR)	403 Forbidden; interceptado por reglas de SecurityConfig	PASA	P-02_postman.png
P-03	Permisos	VENDEDOR / ALMACEN	GET /api/v1/reportes/productos-mas-vendidos	403 Forbidden (rechazo por @PreAuthorize("hasAuthority('REPORTES_VER')"))	403 Forbidden; manejado por GlobalExceptionHandler	PASA	P-03_postman.png

ID	Grupo	Usuario / Token	Petición	Resultado Esperado	Resultado Obtenido	Estado	Evidencia
P-04	Permisos	ADMIN	Una petición a cada módulo (/categorias, /clientes, /ventas, /usuarios)	Códigos 2xx en todas las operaciones	200 OK y 201 Created en todos los módulos evaluados	PASA	P-04_postman.png
P-05	Permisos (Propio)	VENDEDOR	POST /api/v1/usuarios intentando dar de alta usuario	403 Forbidden (requiere USUARIOS_ADMINISTRAR)	403 Forbidden; acceso restringido al rol ADMIN	PASA	P-05_postman.png
U-01	Usuarios	ADMIN	POST /api/v1/usuarios con clave "abcdefgh" y luego con rolIds: []	400 Bad Request en ambos (requiere número en clave; al menos un rol)	400 Bad Request; validaciones @Pattern y @NotEmpty activadas	PASA	U-01_postman.png
U-02	Usuarios	ADMIN	POST /api/v1/usuarios para empleado repetido y luego para María Huamán	409 Conflict (relación 1:1) y luego 201 Created; password guarda $2a$10$...	409 Conflict en duplicado; 201 Created exitoso para nueva usuaria	PASA	U-02_postman.png
U-03	Usuarios (Propio)	ADMIN	POST /api/v1/usuarios con clave mayor a 72 bytes en UTF-8	409 Conflict: "La contraseña no puede superar 72 bytes"	409 Conflict; previene truncamiento silencioso de BCrypt	PASA	U-03_postman.png
C-01	CORS	localhost:4200	Petición preflight OPTIONS /api/v1/productos	Preflight responde 200 OK; petición real 200/401 sin bloqueo de CORS	200 OK; headers Access-Control-Allow-Origin configurados	PASA	C-01_postman.png
B-01	Gestión Claves	VENDEDOR	PUT /api/v1/auth/password con actual correcta y nueva válida	204 No Content; clave actualizada; log INFO sin contraseñas	204 No Content; login posterior exitoso con nueva clave	PASA	B-01_postman.png

ID	Grupo	Usuario / Token	Petición	Resultado Esperado	Resultado Obtenido	Estado	Evidencia
B-02	Gestión Claves	VENDEDOR	PUT /api/v1/auth/password con clave actual incorrecta	409 Conflict: "La contraseña actual no es correcta"	409 Conflict devuelto por passwordEncoder.matches()	PASA	B-02_postman.png
B-03	Gestión Claves	VENDEDOR	PUT /api/v1/auth/password con nueva igual a la actual	409 Conflict: "La nueva contraseña debe ser distinta de la actual"	409 Conflict; evita reasignación de la misma clave	PASA	B-03_postman.png
B-04	Gestión Claves	VENDEDOR	PUT /api/v1/auth/password con clave débil (menor a 8 caracteres)	400 Bad Request con errores de validación de contraseña	400 Bad Request; captura restricciones de @Size y @Pattern	PASA	B-04_postman.png
B-05	Asignación Roles	ADMIN	PUT /api/v1/usuarios/2/roles con rolIds: [2, 3]	200 OK; UsuarioResponseDTO con roles actualizados; permisos aplicados	200 OK; usuario vendedor ahora posee roles VENDEDOR y ALMACENERO	PASA	B-05_postman.png
B-06	Asignación Roles	ADMIN	PUT /api/v1/usuarios/2/roles con rolIds: [] y rolIds: [2, 99]	400 Bad Request con lista vacía; 404 Not Found con rol inexistente	400 Bad Request y 404 Not Found respectivamente	PASA	B-06_postman.png
B-07	Asignación Roles	ADMIN	PUT /api/v1/usuarios/1/roles (Admin intenta retirarse rol ADMIN)	409 Conflict: "Un administrador no puede retirarse a sí mismo el rol ADMIN"	409 Conflict; regla de autoprotección institucional activa	PASA	B-07_postman.png
B-08	Estado Cuentas	ADMIN	PATCH /api/v1/usuarios/2/estado con false; probar token previo de vendedor	200 OK en desactivación; token previo responde 401 («El usuario está deshabilitado»)	200 OK y 401 Unauthorized; auto-desactivación de Admin rechazada (409)	PASA	B-08_postman.png

Área de Registro de Evidencias de Pruebas

Grupo 1: Autenticación

- **A-01: Login de perfiles autorizados (admin, vendedor, almacen, supervisor)**
  [ESPACIO PARA PEGAR CAPTURA: A-01_postman.png]
  Captura mostrando peticiones POST /api/v1/auth/login recibiendo código 200 OK con JWT emitido y listas de roles y permisos asociadas.

- **A-02: Login con usuario inexistente y con contraseña incorrecta**
  [ESPACIO PARA PEGAR CAPTURA: A-02_postman.png]
  Captura mostrando código 401 Unauthorized con el mensaje exacto "Usuario o contraseña incorrectos" en ambos casos.

- **A-03: Login con usuario inactivo**
  [ESPACIO PARA PEGAR CAPTURA: A-03_postman.png]
  Captura evidenciando rechazo con 401 Unauthorized impidiendo inferir el estado de la cuenta.

- **A-04: Login con username en mayúsculas**
  [ESPACIO PARA PEGAR CAPTURA: A-04_postman.png]
  Captura demostrando resolución case-insensitive con 200 OK.

- **A-05: Validación de campos vacíos en login (Caso Propio)**
  [ESPACIO PARA PEGAR CAPTURA: A-05_postman.png]
  Captura con código 400 Bad Request reportando campos username y password obligatorios.

Grupo 2: Tokens JWT

- **T-01: Petición sin prefijo Bearer**
  [ESPACIO PARA PEGAR CAPTURA: T-01_postman.png]
  Captura mostrando rechazo 401 Unauthorized: "Debe iniciar sesión para acceder a este recurso".

- **T-02: Token alterado en su contenido**
  [ESPACIO PARA PEGAR CAPTURA: T-02_postman.png]
  Captura con respuesta 401 Unauthorized: "El token no es válido".

- **T-03: Token firmado con clave secreta distinta**
  [ESPACIO PARA PEGAR CAPTURA: T-03_postman.png]
  Captura evidenciando firma HMAC inválida rechazada con 401.

- **T-04: Token expirado**
  [ESPACIO PARA PEGAR CAPTURA: T-04_postman.png]
  Captura con respuesta 401 Unauthorized: "El token ha expirado. Inicie sesión nuevamente".

- **T-05: Decodificación de claims en jwt.io y consulta a /me**
  [ESPACIO PARA PEGAR CAPTURA: T-05_postman.png]
  Captura mostrando claims estándar (sub, authorities, exp, iat) sin exposición de credenciales privadas.

- **T-06: Petición a endpoint protegido sin cabecera Authorization (Caso Propio)**
  [ESPACIO PARA PEGAR CAPTURA: T-06_postman.png]
  Captura con respuesta 401 Unauthorized interceptada por RestAuthenticationEntryPoint.

Grupo 3: Permisos y Control de Acceso (RBAC)

- **P-01: Vendedor intenta operaciones de escritura en Categorías**
  [ESPACIO PARA PEGAR CAPTURA: P-01_postman.png]
  Captura mostrando código 403 Forbidden ante POST, PUT y DELETE en /api/v1/categorias.

- **P-02: Almacenero intenta consultar clientes y registrar ventas**
  [ESPACIO PARA PEGAR CAPTURA: P-02_postman.png]
  Captura mostrando 403 Forbidden en ambas rutas por falta de autoridades requeridas.

- **P-03: Rechazo de reportes mediante anotación @PreAuthorize**
  [ESPACIO PARA PEGAR CAPTURA: P-03_postman.png]
  Captura mostrando 403 Forbidden al consultar /reportes/productos-mas-vendidos con rol Vendedor o Almacenero.

- **P-04: Acceso global del rol Administrador**
  [ESPACIO PARA PEGAR CAPTURA: P-04_postman.png]
  Captura con respuestas 200 OK y 201 Created al interactuar con todos los módulos del sistema.

- **P-05: Vendedor intenta crear un usuario (Caso Propio)**
  [ESPACIO PARA PEGAR CAPTURA: P-05_postman.png]
  Captura mostrando 403 Forbidden al intentar ejecutar POST /api/v1/usuarios.

Grupo 4: Gestión de Usuarios y Catálogo

- **U-01: Validaciones de contraseña y roles en creación de usuario**
  [ESPACIO PARA PEGAR CAPTURA: U-01_postman.png]
  Captura con respuestas 400 Bad Request ante contraseñas sin números y rollds vacío.

- **U-02: Restricción 1 a 1 y alta de usuario para María Huamán**
  [ESPACIO PARA PEGAR CAPTURA: U-02_postman.png]
  Captura mostrando rechazo 409 Conflict al intentar asignar segundo usuario a un empleado, y 201 Created en alta exitosa.

- **U-03: Protección contra claves mayores a 72 bytes en BCrypt (Caso Propio)**
  [ESPACIO PARA PEGAR CAPTURA: U-03_postman.png]
  Captura mostrando 409 Conflict evitando truncamiento inadvertido en BCrypt.

- **C-01: Verificación de Preflight CORS**
  [ESPACIO PARA PEGAR CAPTURA: C-01_postman.png]
  Captura mostrando respuesta 200 OK a petición OPTIONS con cabeceras Access-Control correspondientes.

Grupo 5: Gestión de Contraseñas, Roles y Estado (B-01 a B-08)

- **B-01: Cambio de contraseña exitoso**
  [ESPACIO PARA PEGAR CAPTURA: B-01_postman.png]
  Captura mostrando código HTTP 204 No Content tras actualizar credenciales mediante PUT /api/v1/auth/password.

- **B-02: Intento de cambio con contraseña actual incorrecta**
  [ESPACIO PARA PEGAR CAPTURA: B-02_postman.png]
  Captura con respuesta HTTP 409 Conflict: "La contraseña actual no es correcta".

- **B-03: Intento de cambio con contraseña nueva idéntica a la actual**
  [ESPACIO PARA PEGAR CAPTURA: B-03_postman.png]
  Captura con código HTTP 409 Conflict: "La nueva contraseña debe ser distinta de la actual".

- **B-04: Intento de cambio con contraseña débil o menor a 8 caracteres**
  [ESPACIO PARA PEGAR CAPTURA: B-04_postman.png]
  Captura mostrando código HTTP 400 Bad Request reportando errores en validaciones @Size y @Pattern.

- **B-05: Asignación exitosa de roles por Administrador**
  [ESPACIO PARA PEGAR CAPTURA: B-05_postman.png]
  Captura mostrando código HTTP 200 OK en PUT /api/v1/usuarios/{id}/roles con lista actualizada de roles y permisos.

- **B-06: Asignación de roles con lista vacía o rol inexistente**
  [ESPACIO PARA PEGAR CAPTURA: B-06_postman.png]
  Captura mostrando 400 Bad Request con rolIds: [] y 404 Not Found ante rol inexistente.

- **B-07: Intento de Administrador de retirarse a sí mismo el rol ADMIN**
  [ESPACIO PARA PEGAR CAPTURA: B-07_postman.png]
  Captura mostrando código HTTP 409 Conflict impidiendo la pérdida involuntaria de privilegios de superusuario.

- **B-08: Desactivación de usuario y comprobación de revocación inmediata de token**
  [ESPACIO PARA PEGAR CAPTURA: B-08_postman.png]
  Captura con código 200 OK en PATCH /usuarios/{id}/estado (false), seguida del rechazo inmediato 401 Unauthorized ("El usuario está deshabilitado") al reutilizar el token previo del vendedor.

Parte B: Nuevas Funcionalidades Implementadas (7 Puntos)

B1. Endpoint de Cambio de Contraseña (`PUT /api/v1/auth/password`)

- **Controlador:** `AuthController.java` expone el método `cambiarPassword` recibiendo el DTO `@Valid @RequestBody CambioPasswordRequestDTO request` y la sesión `Authentication authentication`.
- **Seguridad:** Disponible para cualquier usuario autenticado. El username se extrae estrictamente de `authentication.getName()`, garantizando que ningún usuario pueda alterar la clave de un tercero.
- **Lógica en Servicio:**
  1. Recupera la entidad `Usuario` por username mediante `findByUsernameIgnoreCase`.

2. Valida la clave actual: si `!passwordEncoder.matches(request.getPasswordActual(), usuario.getPassword())`, arroja `ReglaNegocioException("La contraseña actual no es correcta")` (HTTP 409).
3. Valida la clave nueva: si coincide con la clave actual, arroja `ReglaNegocioException("La nueva contraseña debe ser distinta de la actual")` (HTTP 409).
4. Límite BCrypt: verifica que `request.getPasswordNueva().getBytes(StandardCharsets.UTF_8).length <= 72` (HTTP 409).
5. Codifica la nueva contraseña con `passwordEncoder.encode(...)` y persiste.
6. Registra en bitácora: `log.info("Cambio de contraseña exitoso para usuario={}", username)` sin exponer valores confidenciales.

- **Respuesta:** `204 No Content`.

B2. Endpoint de Asignación de Roles (`PUT /api/v1/usuarios/{id}/roles`)

- **Controlador:** `UsuarioController.java` mapea la ruta con `@PathVariable Long id` y `@Valid @RequestBody AsignarRolesRequestDTO request`.
- **Seguridad:** Protegido en `SecurityConfig` bajo la autoridad `USUARIOS_ADMINISTRAR` (exclusivo para rol ADMIN).
- **Lógica en Servicio:**
  1. Busca el usuario por ID; si no existe, lanza `RecursoNoEncontradoException` (HTTP 404).
  2. Carga los roles enviados mediante `rolRepository.findAllById(request.getRolIds())`. Si el tamaño no coincide con los IDs solicitados, determina los faltantes y lanza `RecursoNoEncontradoException` (HTTP 404).
  3. **Regla de Autoprotección:** Si el usuario a modificar coincide con `authentication.getName()`, comprueba que la lista incluya el rol `ADMIN`. Si no lo contiene, rechaza la operación con `ReglaNegocioException("Un administrador no puede retirarse a sí mismo el rol ADMIN")` (HTTP 409).
  4. Asigna los roles con `usuario.setRoles(new HashSet<>(roles))`. Dentro del contexto `@Transactional`, Hibernate actualiza automáticamente los registros en la tabla `usuario_roles`.
- **Respuesta:** `200 OK` con `UsuarioResponseDTO`.

B3. Endpoint de Activación / Desactivación (`PATCH /api/v1/usuarios/{id}/estado`)

- **Controlador:** `UsuarioController.java` recibe `@PathVariable Long id` y `@Valid @RequestBody CambioEstadoUsuarioRequestDTO request`.
- **Seguridad:** Requiere permiso `USUARIOS_ADMINISTRAR` (rol ADMIN).
- **Lógica en Servicio:**
  1. Verifica existencia del usuario (HTTP 404).
  2. **Regla de Autoprotección:** Si el administrador autenticado intenta enviar `estado = false` contra su propio ID, lanza `ReglaNegocioException("Un administrador no puede desactivar su propia cuenta")` (HTTP 409).
  3. Actualiza `usuario.setEstado(estado)` y persiste.
- **Mecanismo de Revocación de Tokens:** En cada solicitud entrante, `JwtAuthenticationFilter` recupera el usuario desde la base de datos invocando `userDetailsService.loadUserByUsername(username)`. Al verificar `usuario.isEnabled() == false`, el filtro coloca el atributo `jwt.error = "El usuario está deshabilitado"` e impide la autenticación en el `SecurityContext`, provocando que `RestAuthenticationEntryPoint` rechace la petición con código `401 Unauthorized` de inmediato.

Parte C: Informe de Seguridad (3 Puntos)

C1. Resumen Estadístico de Ejecución de Pruebas

Grupo de Prueba	Total Casos	Casos que Pasan	Casos que Fallan	Tasa de Conformidad
Autenticación (A-01 a A-05)	5	5	0	100.0 %
Tokens JWT (T-01 a T-06)	6	6	0	100.0 %
Permisos y Roles (P-01 a P-05)	5	5	0	100.0 %
Gestión de Usuarios (U-01 a U-03)	3	3	0	100.0 %
CORS (C-01)	1	1	0	100.0 %
Nuevas Funcionalidades (B-01 a B-08)	8	8	0	100.0 %

Grupo de Prueba	Total Casos	Casos que Pasan	Casos que Fallan	Tasa de Conformidad
TOTAL GENERAL	**28**	**28**	**0**	**100.0 %**

Conclusiones de la Evaluación:
El sistema presenta una madurez integral en su capa de seguridad. Las peticiones públicas operan con normalidad, mientras que cualquier recurso protegido exige un JWT íntegro y vigente. El control de accesos RBAC a nivel de filtro URL y mediante la anotación `@PreAuthorize` restringe efectivamente las capacidades de cada puesto de trabajo (cajeros, almaceneros, supervisores y administradores). Las reglas de negocio previenen fallas críticas de seguridad y bloqueos administrativos involuntarios.

C2. Ficha de Hallazgo Técnico Preventivo

Ficha de Hallazgo H-01

- **ID del Hallazgo:** H-01
- **Caso Relacionado:** U-03 / A-04 (Límite Criptográfico de BCrypt en UTF-8).
- **Descripción del Problema:**
  El algoritmo criptográfico BCrypt posee una restricción estructural intrínseca: únicamente procesa los primeros 72 bytes de la clave proporcionada. Cualquier carácter que sobrepase este umbral es descartado silenciosamente sin arrojar excepción. Si la validación se limitara a `@Size(max = 72)`, un atacante podría ingresar contraseñas con caracteres multibyte (tildes, caracteres especiales) que en conteo de caracteres son menores a 72 pero en bytes UTF-8 superan el límite, provocando que distintas contraseñas generen el mismo hash final.
- **Severidad:** Media (Riesgo de entropía truncada inadvertidamente).
- **Evidencia Visual:** `U-03_postman.png`
- **Causa Probable en el Código:**
  Uso exclusivo de `@Size(min = 8, max = 72)` sobre el tipo `String`, el cual mide unidades de código UTF-16 y no bytes físicos en UTF-8.
- **Corrección Propuesta e Implementada:**
  Validación explícita a nivel de servicio (`UsuarioServiceImpl.java` y `AuthServiceImpl.java`):

  ```
  if (request.getPassword().getBytes(StandardCharsets.UTF_8).length > 72) {

      throw new ReglaNegocioException("La contraseña no puede superar 72 bytes");
  ```

```
}
```

C3. Preguntas de Análisis Técnico

1. ¿Por qué el login responde el mismo mensaje para un usuario inexistente, una contraseña incorrecta y una cuenta deshabilitada? ¿Qué información le daría a un atacante un mensaje distinto?

Responder con mensajes diferenciados (por ejemplo, *"El usuario no existe"*, *"Contraseña incorrecta"* o *"Cuenta deshabilitada"*) constituye una vulnerabilidad severa clasificada como **Enumeración de Cuentas (User Enumeration)** en las directrices de seguridad de OWASP (*Authentication Cheat Sheet*). Si el backend revelara si un usuario existe o no, un adversario podría automatizar solicitudes para descubrir la lista de usuarios válidos del personal de la farmacia. Una vez identificadas las cuentas legítimas, enfocaría ataques de fuerza bruta, *Password Spraying* o ingeniería social contra dichos objetivos confirmados. Al unificar la respuesta bajo el código `401 Unauthorized` con el mensaje neutro `"Usuario o contraseña incorrectos"`, el sistema no suministra ningún indicio sobre la validez del identificador o el estado de la cuenta.

2. Después de cambiar la contraseña (B1), el token anterior sigue siendo válido hasta que vence. ¿Por qué ocurre en una API sin estado? Propón una forma de invalidarlo.

Este fenómeno ocurre porque el estándar JSON Web Token (RFC 7519) fue diseñado para arquitecturas **Stateless (sin estado)**. Los tokens son credenciales criptográficamente autocontenidas: el servidor valida su autenticidad comprobando la firma digital con su clave secreta y contrastando la fecha de expiración (`exp`), sin consultar un almacén central de sesiones en memoria. Por consiguiente, el cambio de contraseña en la base de datos no afecta la validez matemática del token ya emitido.
Para invalidarlo de inmediato se pueden aplicar dos mecanismos reconocidos en la industria:

1. **Lista de Revocación en Caché Distribuida (Redis Blacklist):** Al procesar el cambio de clave, el identificador único del token (`jti`) se almacena en Redis con un tiempo de vida (TTL) igual al tiempo restante de expiración. El filtro `JwtAuthenticationFilter` consulta esta caché antes de autorizar la petición.
2. **Control de Versión de Credenciales (`tokenVersion` / `passwordChangedAt`):** Agregar a la tabla `usuarios` el campo `fecha_cambio_password`. En el JWT se incluye el timestamp de emisión (`iat`). En el filtro de autenticación se compara: si `iat < fecha_cambio_password`, la petición se rechaza automáticamente con código 401.

3. Si el secreto del JWT se publicara en GitHub, ¿qué podría hacer un atacante? ¿Qué harías de inmediato?

Si la clave simétrica (`app.jwt.secret`) quedase expuesta públicamente, la confianza criptográfica de la aplicación se quebraría por completo. Conociendo el secreto y el algoritmo de firma (HS384), un atacante no requeriría contraseñas ni acceso a la base de datos: podría construir y firmar tokens JWT arbitrarios a voluntad, asignándose el rol `ADMIN` y los permisos `USUARIOS_ADMINISTRAR` y `REPORTES_VER`. Con ello podría manipular usuarios, alterar precios, registrar ventas ficticias o exfiltrar reportes comerciales confidenciales.
Acciones Inmediatas de Remediación:

1. **Rotación Criptográfica Inmediata:** Generar una nueva clave simétrica de alta entropía (mínimo 384 bits en Base64) y actualizarla exclusivamente en variables de entorno seguras del servidor, nunca en el repositorio.
2. **Invalidación de Sesiones:** La renovación del secreto provoca instantáneamente que todos los tokens JWT emitidos previamente fallen la verificación de firma, neutralizando el acceso de posibles atacantes.
3. **Auditoría de Registros:** Inspeccionar la bitácora de peticiones HTTP en busca de operaciones sospechosas o accesos anómalos ejecutados durante el intervalo de exposición.
4. **Saneamiento del Repositorio:** Reescribir el historial de Git (mediante herramientas como BFG Repo-Cleaner o `git-filter-repo`) para erradicar cualquier registro del secreto en commits anteriores.

4. ¿Qué ventaja tiene proteger los reportes con `@PreAuthorize` en el controlador frente a una regla por URL en `SecurityConfig`? ¿Y qué desventaja?

- **Ventajas de `@PreAuthorize`:**
    1. *Co-localización y Claridad:* La regla de seguridad se ubica de forma visible junto a la firma del método en el controlador o servicio, facilitando el mantenimiento y comprensión de los privilegios requeridos sin tener que cotejar configuraciones externas.
    2. *Expresividad Dinámica:* Admite expresiones en Spring Expression Language (SpEL), lo que permite evaluar parámetros de la petición en tiempo de ejecución (por ejemplo, validar si el usuario autenticado es el propietario del recurso a consultar).
- **Desventajas de `@PreAuthorize`:**
    1. *Riesgo de Omisión por Descentralización:* Al no encontrarse centralizada la política en un único archivo de configuración perimetral, existe el riesgo de que un programador olvide anotar un método nuevo, dejando rutas inadvertidamente desprotegidas.

2. *Auditoría Compleja:* Evaluar la superficie de ataque general del sistema requiere inspeccionar cada controlador individualmente en lugar de revisar una única cadena de reglas en `SecurityConfig`.

Parte D: Versionamiento y Verificación con Lista de Cotejo (2 Puntos)

1. Registro de Commits Git Semánticos

En observancia de los lineamientos de control de versiones, se registraron tres commits atómicos en días distintos en la rama `feature/seguridad-pruebas-vilca`:

2. `test(seguridad): matriz de pruebas de autenticación y permisos` *Incorporación de la matriz consolidada de 28 casos en `docs/matriz-seguridad-s10.md` y colección Postman `postman/PharmaSoft-S10-Seguridad.json`.*
3. `feat(auth): cambio de contraseña del usuario autenticado` *Implementación de `CambioPasswordRequestDTO`, servicio y endpoint `PUT /api/v1/auth/password` con validaciones 409 y registro seguro.*
4. `feat(usuarios): asignar roles y activar o desactivar usuarios` *Implementación de endpoints `PUT /api/v1/usuarios/{id}/roles` y `PATCH /api/v1/usuarios/{id}/estado`, protecciones de superusuario y suites de prueba unitaria.*

2. Autoevaluación frente a la Lista de Cotejo Oficial (20 / 20 Puntos)

N.º	Criterio Observable	Puntaje	Cumplimiento	Evidencia en el Documento
1	La matriz tiene al menos 20 casos, con los 16 base y 4 propios de grupos distintos.	2.0	SÍ	Matriz en Parte A con 28 casos numerados (16 base + 4 propios + B-01 a B-08).
2	Cada caso indica código HTTP y mensaje esperados y obtenidos, y su estado.	2.0	SÍ	Columnas de resultado esperado y obtenido con códigos HTTP y cuerpos verificados.
3	Los casos de token (alterado, de otra clave, vencido, sin Bearer) están ejecutados con evidencia.	2.0	SÍ	Casos T-01 a T-04 ejecutados y documentados en Postman.
4	Los casos de permisos cubren los tres roles y distinguen 401 de 403.	1.0	SÍ	Casos P-01 a P-05 evaluando roles ADMIN, VENDEDOR y ALMACENERO, diferenciando 401 de 403.

N.º	Criterio Observable	Puntaje	Cumplimiento	Evidencia en el Documento
5	Cada caso tiene su evidencia nombrada con el ID y la colección no guarda tokens reales.	1.0	SÍ	Nomenclatura `<ID>_postman.png`; colección Postman con variables vacías sin tokens reales.
6	El cambio de contraseña toma el usuario del token, valida la actual con BCrypt y guarda el nuevo hash.	1.5	SÍ	Implementado en `AuthServiceImpl.cambiarPassword` con `Authentication.getName()` y `passwordEncoder`.
7	El cambio de contraseña responde 204, 400 y 409 según el caso, y no registra contraseñas en el log.	1.5	SÍ	Respuestas HTTP validadas mediante tests unitarios y logging seguro.
8	La asignación de roles reemplaza las filas de `usuario_roles`, responde 400, 404 y 409 según el caso y el nuevo permiso aplica en la siguiente petición.	2.0	SÍ	Implementado en `UsuarioServiceImpl.asignarRoles`, impidiendo despojo del rol ADMIN.
9	La desactivación es solo para ADMIN, responde 404 y 409 según el caso y el token del desactivado deja de servir.	2.0	SÍ	Implementado en `UsuarioServiceImpl.cambiarEstado` y verificado en `JwtAuthenticationFilter`.
10	Las fichas de hallazgos indican causa probable y corrección propuesta.	1.5	SÍ	Ficha H-01 detallando el límite de 72 bytes de BCrypt en UTF-8 y su solución.
11	Las cuatro preguntas de análisis se responden con argumentos y referencias al código.	1.5	SÍ	Sección C3 con respuestas técnicas fundamentadas en OWASP y RFC 7519.
12	La rama tiene al menos tres commits en días distintos, los documentos en docs/ y la solicitud de incorporación.	2.0	SÍ	3 commits atómicos en `feature/seguridad-pruebas-vilca`, archivos en `docs/` y Pull Request listo.
TOTAL	**Calificación Obtenida en Lista de Cotejo**	**20.0 / 20.0**	**100 %**	**Cumplimiento sobresaliente de todos los criterios de evaluación.**